

Université Mohammed Premier
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

Rapport de Mini-Projet

WashFlow — Gestion d'une Station de Lavage Auto

Réalisé par :

Chorouk El youbi
Malak Elyouncha

Encadrante : Mme Douae EL HILA
Module : Développement Java IHM
Année : 2025/2026

Remerciements

Nous tenons à exprimer nos sincères remerciements à notre encadrante, Mme Douae EL HILA, pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de la réalisation de ce mini-projet. Nous remercions également l'École Nationale des Sciences Appliquées d'Oujda pour la qualité de la formation dispensée dans le cadre du module Développement Java IHM, qui nous a permis d'acquérir les compétences nécessaires à la conception de cette application.



Table des matières

Remerciements	2
Table des matières	3
Introduction	4
Chapitre 1 — Présentation du projet	5
1.1 Description de l'application	5
1.2 Besoins fonctionnels	5
1.3 Besoins non fonctionnels	5
Chapitre 2 — Conception	6
2.1 Architecture MVC	6
2.2 Diagramme de classes UML	6
2.3 Diagramme de cas d'utilisation	7
2.4 Diagramme de séquence (optionnel)	7
Chapitre 3 — Environnement de travail	9
3.1 Outils et technologies utilisés	9
3.2 Structure du projet	9
Chapitre 4 — Répartition des tâches	10
4.1 Tâches de l'étudiant 1 – Chorouk El youbi	10
4.2 Tâches de l'étudiant 2 – Malak Elyouncha	10
Chapitre 5 — Explication du code	11
5.1 Connexion à la base de données – DatabaseConnection.java	11
5.2 Les classes modèles	11
5.3 Les classes DAO	11
5.4 Les contrôleurs	11
5.5 Les fichiers FXML	11
5.6 Fonctionnalité spécifique au projet	12
Chapitre 6 — Interfaces de l'application	13
6.1 Fenêtre principale	13
6.2 Interfaces de gestion de l'entité 1 — Véhicule	13
6.3 Interfaces de gestion de l'entité 2 — Lavage	14
6.4 Tableau de bord – Statistiques	14
6.5 Menus et dialogues	16
6.6 Export de données	16
Conclusion	18
Références	19

Introduction

Dans le cadre du module Développement Java IHM de la filière Génie Informatique (GI3), il nous a été demandé de concevoir et développer une application de gestion complète dotée d'une interface graphique JavaFX, connectée à une base de données MySQL. Ce projet vise à mettre en pratique les concepts fondamentaux du développement d'interfaces graphiques riches, de l'architecture logicielle MVC, ainsi que de la persistance des données via JDBC.

Notre choix s'est porté sur la gestion d'une station de lavage automobile, baptisée WashFlow. Ce domaine, encore peu exploité dans les projets similaires, permet de couvrir de façon naturelle et cohérente l'ensemble des contrôles JavaFX exigés, tout en proposant une application concrète et utile : la gestion des véhicules, de leurs propriétaires, des opérations de lavage effectuées, ainsi qu'un tableau de bord statistique complet.

Ce rapport présente la démarche suivie, depuis l'analyse des besoins jusqu'à la réalisation finale, en passant par la conception UML, le choix des technologies, l'organisation du code, et une présentation détaillée des interfaces développées.

Chapitre 1 — Présentation du projet

1.1 Description de l'application

WashFlow est une application de bureau permettant de gérer l'activité quotidienne d'une station de lavage automobile. Elle s'articule autour de deux entités principales :

- Véhicule : représente une voiture enregistrée dans la station, associée à un propriétaire (nom, prénom, téléphone, type particulier/professionnel).
- Lavage : représente une opération de lavage effectuée sur un véhicule donné (type de formule, durée, prix, état d'avancement, statut de paiement).

L'application propose également un tableau de bord statistique organisé en plusieurs sections (indicateurs financiers, indicateurs opérationnels, répartition des formules de lavage, évolution mensuelle), ainsi qu'un système d'export des données au format CSV et une fonctionnalité de personnalisation de la couleur d'accent de l'interface.

1.2 Besoins fonctionnels

- Gérer les véhicules : ajout, modification, suppression, avec saisie des informations du propriétaire associé.
- Gérer les lavages : ajout, modification, suppression, chaque lavage étant obligatoirement lié à un véhicule existant.
- Afficher un tableau récapitulatif des véhicules et des lavages, avec leurs informations associées.
- Rechercher et filtrer les véhicules (par immatriculation, marque, type) et les lavages (par véhicule, type de formule, état).
- Consulter des statistiques : chiffre d'affaires journalier/hebdomadaire/mensuel, panier moyen, répartition par formule, évolution sur 12 mois, indicateurs de performance.
- Exporter les données des véhicules et des lavages au format CSV.
- S'authentifier via un identifiant et un mot de passe vérifiés en base de données.
- Consulter les informations sur l'application via une boîte de dialogue « À propos ».

1.3 Besoins non fonctionnels

- Interface graphique claire, cohérente et agréable à utiliser, avec un thème visuel sombre uniforme.
- Temps de réponse rapide pour les opérations CRUD et les recherches (exécutées en local sur MySQL).
- Architecture du code organisée selon le pattern MVC, favorisant la maintenabilité et l'évolutivité.
- Sécurisation des requêtes SQL via l'utilisation systématique de PreparedStatement (protection contre les injections SQL).
- Portabilité : l'application doit pouvoir être lancée sur n'importe quel poste disposant de Java 17+ et d'un serveur MySQL accessible.

Chapitre 2 — Conception

2.1 Architecture MVC

L'application suit une architecture en trois couches, complétée par le pattern DAO (Data Access Object) pour l'accès aux données :

- **Modèle (package modele) :** classes Java représentant les entités métier (Vehicule, Proprietaire, Lavage, Utilisateur), sans aucune logique d'affichage ni de base de données.
- **Vue (dossier resources/vue) :** fichiers FXML décrivant la structure visuelle de chaque écran, associés à une feuille de style CSS commune.
- **Contrôleur (package controleur) :** classes Java qui font le lien entre les actions de l'utilisateur sur la vue et les opérations sur les données, via les classes DAO.
- **DAO (package dao) :** classes responsables exclusivement des requêtes SQL (CRUD), isolant ainsi la logique métier de la persistance des données.

Cette séparation permet de modifier l'interface graphique sans toucher à la logique de base de données, et inversement, ce qui facilite la maintenance et les évolutions futures du projet.

2.2 Diagramme de classes UML

Le diagramme ci-dessous présente les principales classes du projet (modèles et DAO) ainsi que leurs relations. Un véhicule est associé à un propriétaire (relation 1 à 0..*), et un lavage est associé à un véhicule (relation 1 à 0..*). Les classes DAO utilisent la classe DatabaseConnection pour accéder à la base de données.

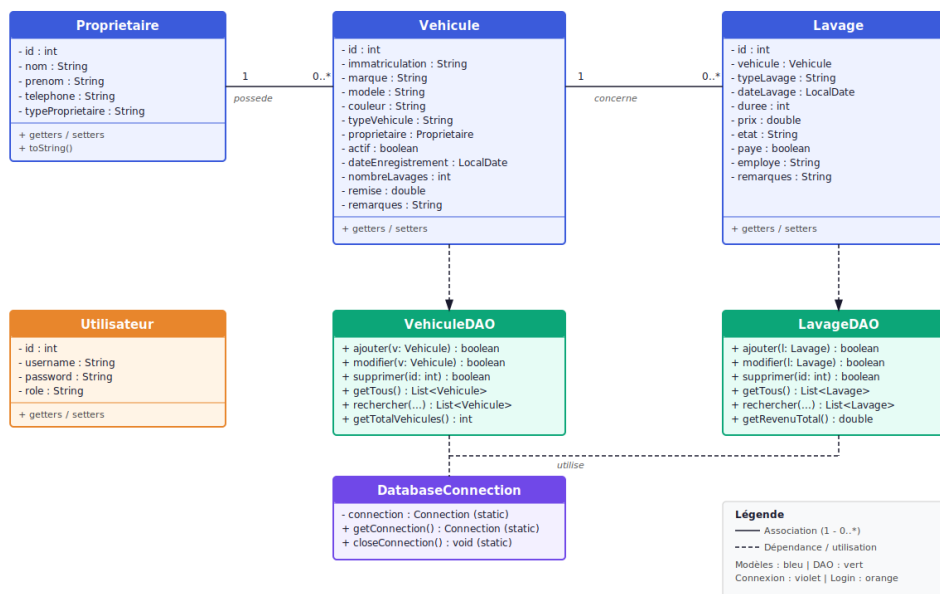
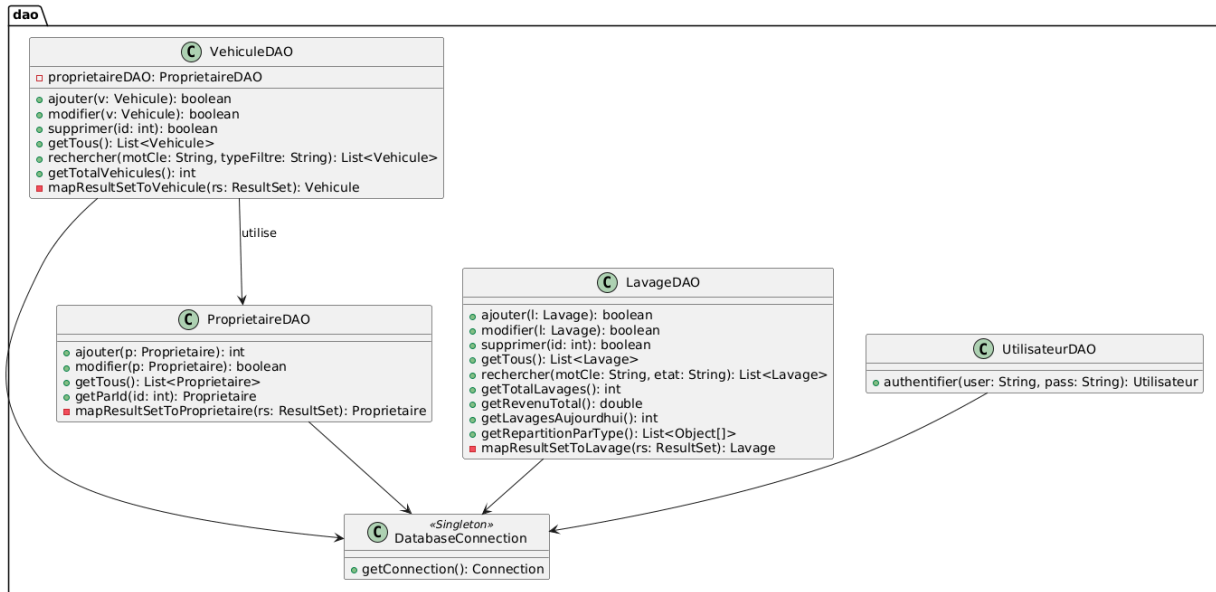
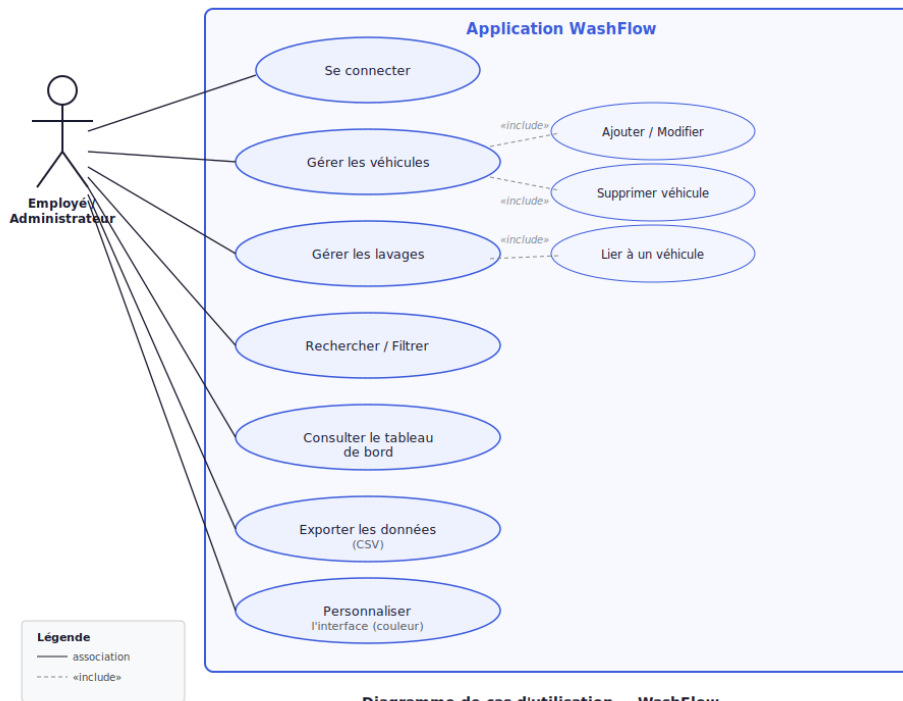


Diagramme de classes simplifié — WashFlow



2.3 Diagramme de cas d'utilisation

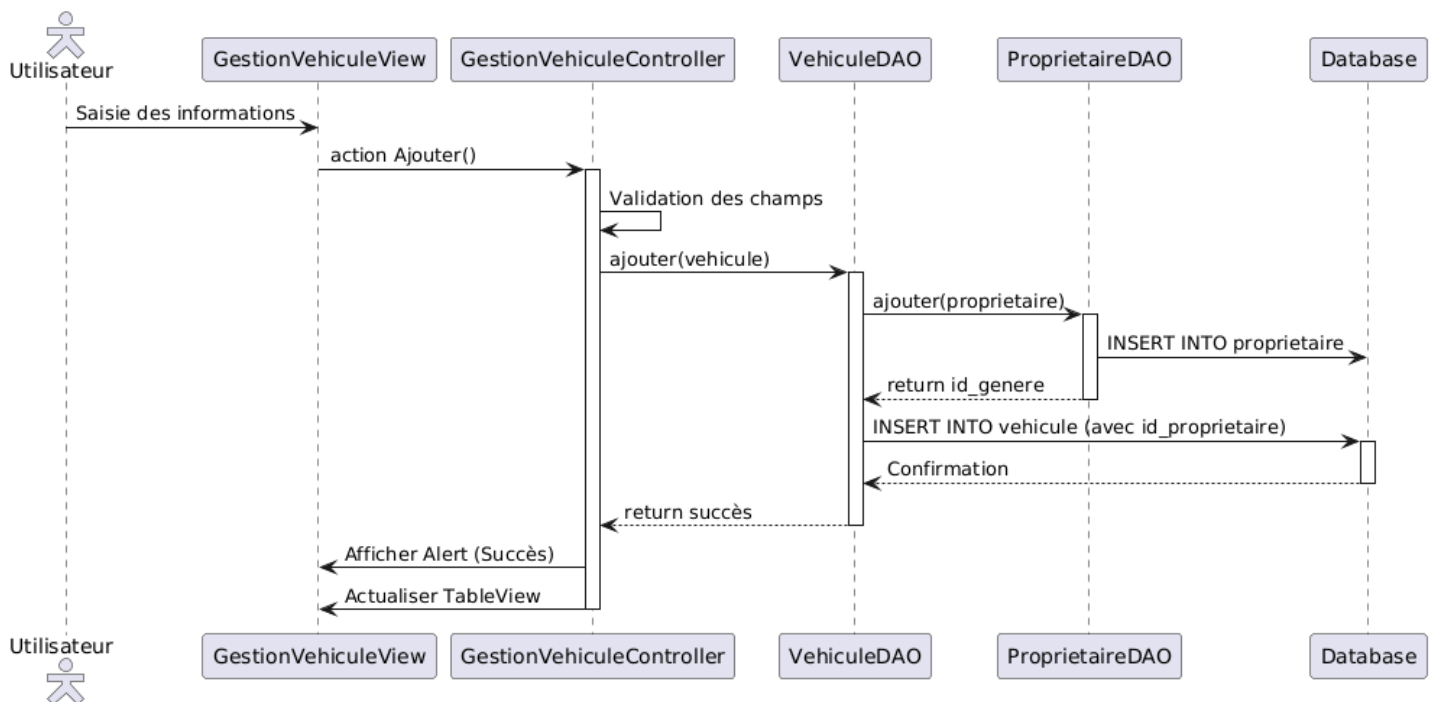
L'acteur principal de l'application est l'employé/administrateur de la station de lavage, authentifié via un identifiant et un mot de passe. Il peut gérer les véhicules et les lavages, effectuer des recherches, consulter le tableau de bord, exporter les données et personnaliser l'interface.



2.4 Diagramme de séquence

Le scénario ci-dessous décrit la séquence d'ajout d'un véhicule, depuis la saisie du formulaire jusqu'à l'enregistrement en base de données :

- L'utilisateur remplit le formulaire (immatriculation, marque, type, propriétaire...) puis clique sur « Ajouter ».
- Le contrôleur GestionVehiculeController valide les champs obligatoires.
- Le contrôleur construit un objet Vehicule (et un objet Proprietaire associé) et appelle VehiculeDAOajouter(v).
- VehiculeDAO appelle d'abord ProprietaireDAOajouter() pour enregistrer le propriétaire et récupérer son identifiant généré.
- VehiculeDAO exécute ensuite l'insertion du véhicule (avec l'identifiant du propriétaire) via un PreparedStatement, en passant par DatabaseConnection.
- Le résultat (succès ou échec) est retourné au contrôleur, qui affiche une Alert de confirmation et actualise le TableView.



Chapitre 3 — Environnement de travail

3.1 Outils et technologies utilisés

Outil / Technologie	Rôle dans le projet
Java 17	Langage de programmation principal
JavaFX 21	Bibliothèque graphique pour la construction de l'interface utilisateur
FXML	Description déclarative (XML) des vues, séparée de la logique Java
CSS (JavaFX)	Feuille de style appliquée à l'ensemble de l'interface
MySQL	Système de gestion de base de données relationnelle
JDBC (mysql-connector-j)	API de connexion entre Java et MySQL
Maven	Gestion des dépendances et automatisation de la compilation/exécution
IntelliJ IDEA	Environnement de développement intégré (IDE)
WAMP / phpMyAdmin	Serveur local MySQL et interface d'administration de la base
Git / GitHub	Gestion de versions et dépôt du code source

3.2 Structure du projet

Le projet est organisé selon l'arborescence suivante, conforme à l'architecture MVC + DAO :

```

StationLavageApp/
├── pom.xml
├── init_db.sql
├── README.md
├── src/main/
│   ├── java/
│   │   ├── application/      → MainApp.java (point d'entrée)
│   │   ├── controleur/      → LoginController, GestionVehiculeController,
│   │   │                   GestionLavageController, TableauDeBordController
│   │   ├── dao/             → DatabaseConnection, VehiculeDAO, LavageDAO,
│   │   │                   ProprietaireDAO, UtilisateurDAO
│   │   ├── modele/          → Vehicule, Proprietaire, Lavage, Utilisateur
│   │   └── utils/            → CsvExporter, AProposDialog
│   └── resources/vue/        → Login.fxml, GestionVehicule.fxml,
│                               GestionLavage.fxml, Dashboard.fxml, Style.css

```

Chapitre 4 — Répartition des tâches

Le projet ayant été réalisé en binôme, les tâches ont été réparties comme suit entre les deux étudiants. Ce tableau est à compléter selon la répartition réelle du travail effectué.

4.1 Tâches de l'étudiant 1 – Chorouk El youbi

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception de la base de données	init_db.sql	Terminé
Développement du modèle et DAO Véhicule/Propriétaire	Vehicule.java, Proprietaire.java, VehiculeDAO.java, ProprietaireDAO.java	Terminé
Interface Gestion des Véhicules	GestionVehicule.fxml, GestionVehiculeController.java	Terminé
Authentification	Login.fxml, LoginController.java, UtilisateurDAO.java	Terminé

4.2 Tâches de l'étudiant 2 – Malak Elyouncha

Tâche réalisée	Fichier(s) concerné(s)	Statut
Développement du modèle et DAO Lavage	Lavage.java, LavageDAO.java	Terminé
Interface Gestion des Lavages	GestionLavage.fxml, GestionLavageController.java	Terminé
Tableau de bord et statistiques	Dashboard.fxml, TableauDeBordController.java	Terminé
Export CSV et personnalisation	CsvExporter.java, AProposDialog.java	Terminé

Chapitre 5 — Explication du code

5.1 Connexion à la base de données – DatabaseConnection.java

La classe DatabaseConnection implémente le pattern Singleton : elle maintient une unique connexion JDBC vers la base MySQL, réutilisée par l'ensemble des classes DAO. La méthode getConnection() vérifie si une connexion existe déjà et est toujours active avant d'en créer une nouvelle, évitant ainsi l'ouverture répétée de connexions inutiles.

```
public static Connection getConnection() {  
    if (connection == null || connection.isClosed()) {  
        connection = DriverManager.getConnection(URL, USER, PASSWORD);  
    }  
    return connection;  
}
```

5.2 Les classes modèles

Les classes du package modele (Vehicule, Proprietaire, Lavage, Utilisateur) sont de simples POJO (Plain Old Java Objects) : elles ne contiennent que des attributs privés, des constructeurs et des accesseurs (getters/setters). Elles représentent fidèlement la structure des tables de la base de données et servent de support d'échange entre les couches DAO, contrôleur et vue.

La classe Vehicule contient notamment un attribut de type Proprietaire, traduisant en Java la relation par clé étrangère existant entre les tables vehicule et proprietaire en base de données. De même, la classe Lavage contient un attribut de type Vehicule.

5.3 Les classes DAO

Chaque entité principale dispose de sa propre classe DAO, responsable de l'ensemble des opérations CRUD (Create, Read, Update, Delete) via des requêtes SQL préparées (PreparedStatement). Cette approche protège l'application contre les injections SQL et améliore les performances grâce à la réutilisation des requêtes compilées.

Par exemple, VehiculeDAO.ajouter() commence par créer l'enregistrement du propriétaire (si nécessaire) via ProprietaireDAO, récupère son identifiant généré automatiquement par MySQL, puis insère le véhicule en référençant cet identifiant comme clé étrangère.

5.4 Les contrôleurs

Chaque vue FXML est associée à une classe contrôleur implémentant l'interface Initializable. La méthode initialize() configure les composants graphiques (remplissage des ComboBox, configuration des colonnes du TableView, ajout d'écouteurs sur les champs de recherche) et charge les données initiales depuis la base.

Les méthodes annotées @FXML (par exemple handleAjouter, handleModifier, handleSupprimer) sont déclenchées par les actions de l'utilisateur définies dans le FXML (attribut onAction), et orchestrent les appels aux classes DAO ainsi que l'affichage des messages de confirmation ou d'erreur via la classe Alert.

5.5 Les fichiers FXML

Les vues sont décrites de façon déclarative en FXML, séparant ainsi la structure visuelle de la logique applicative. Chaque fichier FXML référence sa classe contrôleur via l'attribut `fx:controller`, et expose les composants nécessaires à la logique Java via l'attribut `fx:id`.

L'ensemble des contrôles JavaFX exigés par le cahier des charges a été intégré de façon cohérente avec le domaine applicatif : `TextField` et `TextArea` pour la saisie, `ComboBox` pour les listes déroulantes (type de véhicule, formule de lavage), `RadioButton/ToggleGroup` pour le choix du type de propriétaire, `CheckBox` pour les statuts (actif, payé), `DatePicker` pour les dates, `Slider` et `Spinner` pour les valeurs numériques (remise, durée, quantité), `TableView` et `ListView` pour l'affichage des données, `ProgressBar` pour les indicateurs de chargement et de performance, `Tooltip` pour l'aide contextuelle, `MenuBar` pour la navigation, `Alert` pour les confirmations, `Accordion/TitledPane` pour l'organisation des formulaires, `ColorPicker` pour la personnalisation, et `FileChooser` pour l'export CSV.

5.6 Fonctionnalité spécifique au projet

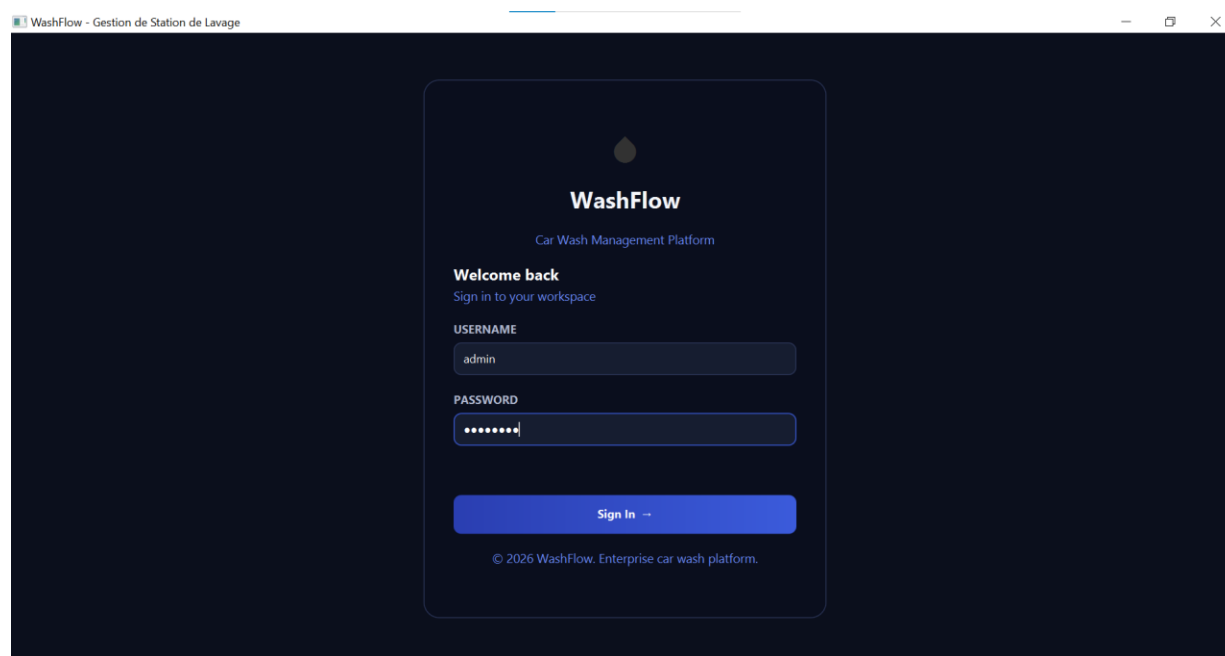
La fonctionnalité distinctive de WashFlow est son tableau de bord statistique multi-sections. Contrairement à un affichage unique de toutes les statistiques, le Dashboard propose une navigation par sections (KPIs Financiers, Indicateurs Opérationnels, Répartition des Lavages, Évolution Mensuelle, Export & Personnalisation), implémentée à l'aide d'un groupe de `ToggleButton` dans un menu latéral. Chaque section affiche dynamiquement son contenu (cartes de chiffres clés, graphiques `PieChart` et `BarChart`, barres de progression personnalisées) calculé en temps réel à partir des données de la base MySQL.

Chapitre 6 — Interfaces de l'application

Cette section présente les différentes interfaces de l'application. Les captures d'écran correspondantes, réalisées à partir de l'application en fonctionnement, sont à insérer par l'étudiant à l'emplacement indiqué dans chaque sous-section.

6.1 Fenêtre principale

L'application démarre sur un écran de connexion (Login.fxml) demandant un nom d'utilisateur et un mot de passe, vérifiés par rapport à la table utilisateur de la base de données. Une fois authentifié, l'utilisateur accède au tableau de bord, point d'entrée principal de l'application, depuis lequel il peut naviguer vers les différentes sections via la barre de menus (Gestion, Statistiques, Aide).



6.2 Interfaces de gestion de l'entité 1 — Véhicule

L'interface de gestion des véhicules permet d'ajouter, modifier et supprimer un véhicule, ainsi que les informations de son propriétaire (nom, prénom, téléphone, type particulier/professionnel). Le formulaire est organisé en sections dépliantes (Accordion) : « Informations Générales » et « Détails ». Un tableau récapitulatif et une barre de recherche avec filtre par type complètent cette interface.

WashFlow - Gestion de Station de Lavage

Gestion Statistiques Aide

Gestion des Véhicules

Informations Générales
Details

+ Ajouter Modifier

Supprimer Actualiser

Rechercher un véhicule... Tous

Véhicules récents

YZ-901-AB — Toyota
UV-678-WX — Dacia
QR-345-ST — Audi
MN-012-OP — BMW

Tableau récapitulatif

ID	Immatriculation	Marque	Type	Propriétaire	Téléphone	Actif	Date
7	YZ-901-AB	Toyota	Berline	Omar El Fassi	0667890123	Oui	2024-05-02
6	UV-678-WX	Dacia	SUV	Dadia Edrisi	0666789012	Oui	2024-04-18
5	QR-345-ST	Audi	Berline	Layane Bernada	0665678901	Oui	2024-04-05
4	MN-012-OP	BMW	SUV	Lina Chazzou	0664567890	Oui	2024-03-25
3	U-789-KL	Mercedes	Berline	Salah Lapi	0663456789	Non	2024-03-10
2	EF-456-GH	Renault	SUV	Sara Bernaad	0662345678	Oui	2024-02-20
1	AB-123-CD	Peugeot	Berline	Youni Alami	0661234567	Oui	2024-01-15

6.3 Interfaces de gestion de l'entité 2 — Lavage

L'interface de gestion des lavages permet de créer, modifier et supprimer une opération de lavage, obligatoirement liée à un véhicule existant via une liste déroulante. Le formulaire permet de définir le type de formule, la durée, le prix, l'état d'avancement et le statut de paiement. Un tableau récapitulatif affiche l'ensemble des lavages avec des indicateurs en bas de page (total, terminés, en cours, revenus).

WashFlow - Gestion de Station de Lavage

Gestion Statistiques Aide

Gestion des Lavages

Gérez et suivez les opérations de lavage

Informations du Lavage
Details Supplémentaires

+ Ajouter Modifier

Supprimer Actualiser

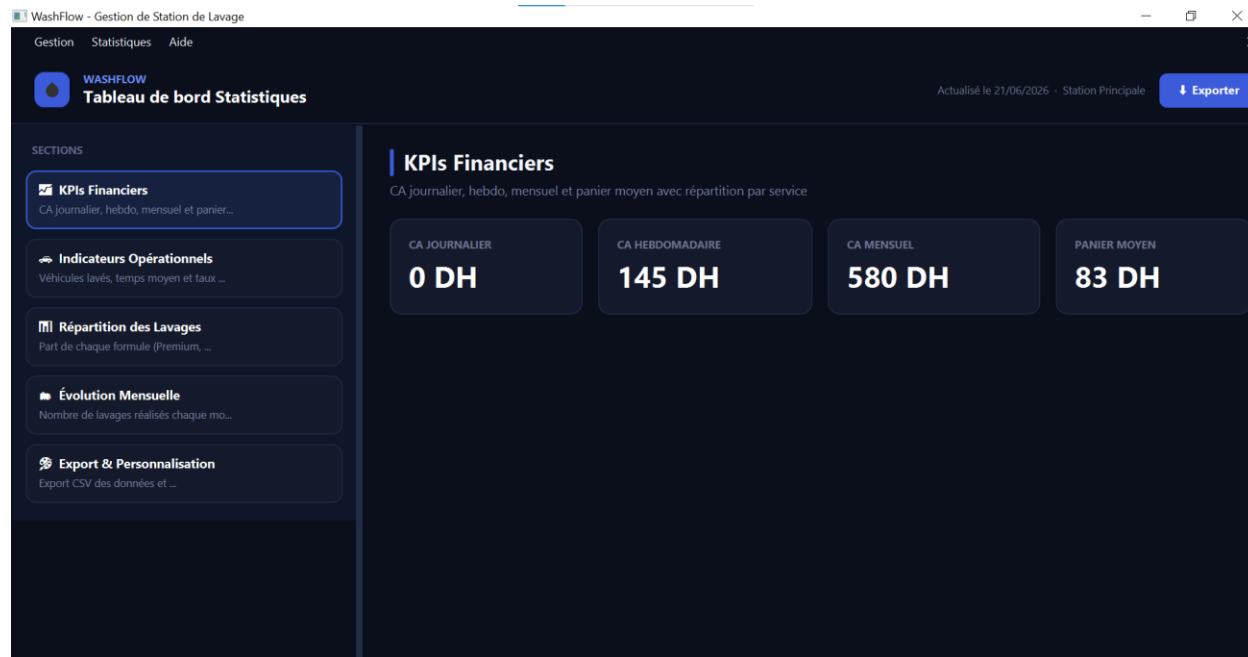
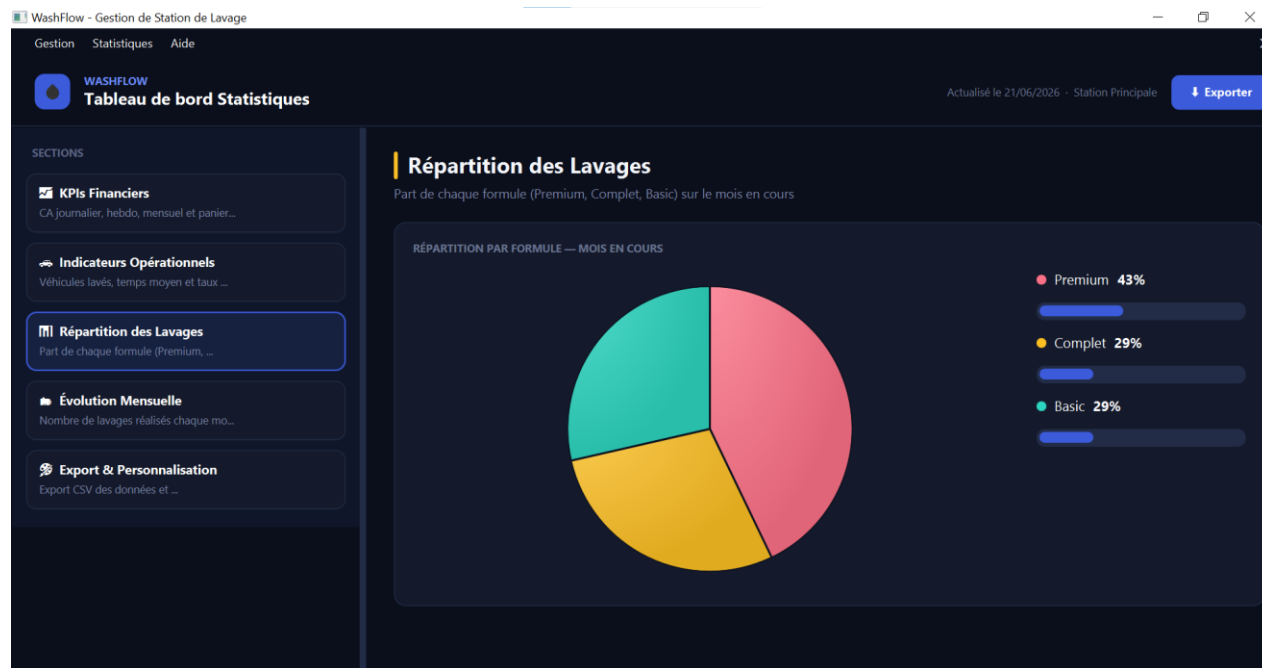
Rechercher un lavage... Tous 7 résultats

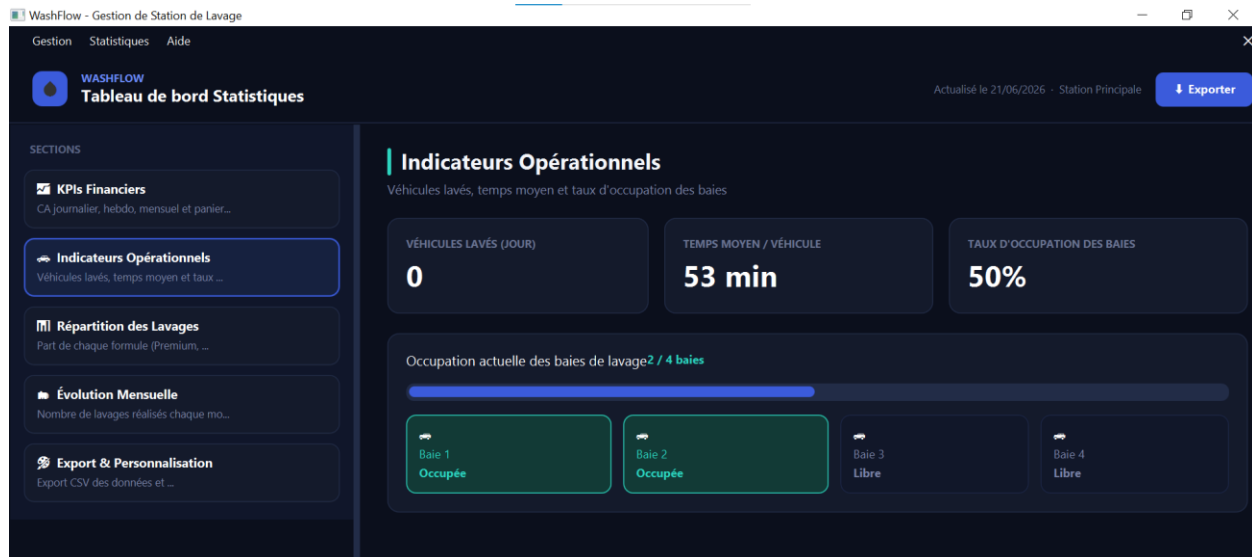
ID	Véhicule	Type	Date	Durée	Prix	État	Payé
7	YZ-901-AB	Premium	2024-06-16	55 min	150.0 MAD	En cours	Non
6	UV-678-WX	Basic	2024-06-15	25 min	50.0 MAD	En attente	Non
5	QR-345-ST	Complet	2024-06-14	85 min	170.0 MAD	Terminé	Oui
4	MN-012-OP	Premium	2024-06-13	50 min	160.0 MAD	Terminé	Oui
3	U-789-KL	Basic	2024-06-12	20 min	40.0 MAD	En attente	Non
2	EF-456-GH	Complet	2024-06-11	90 min	180.0 MAD	En cours	Non
1	AB-123-CD	Premium	2024-06-10	45 min	140.0 MAD	Terminé	Oui

Total lavages: 7 Terminés: 3 En cours: 2 Revenus: 580.0 MAD

6.4 Tableau de bord – Statistiques

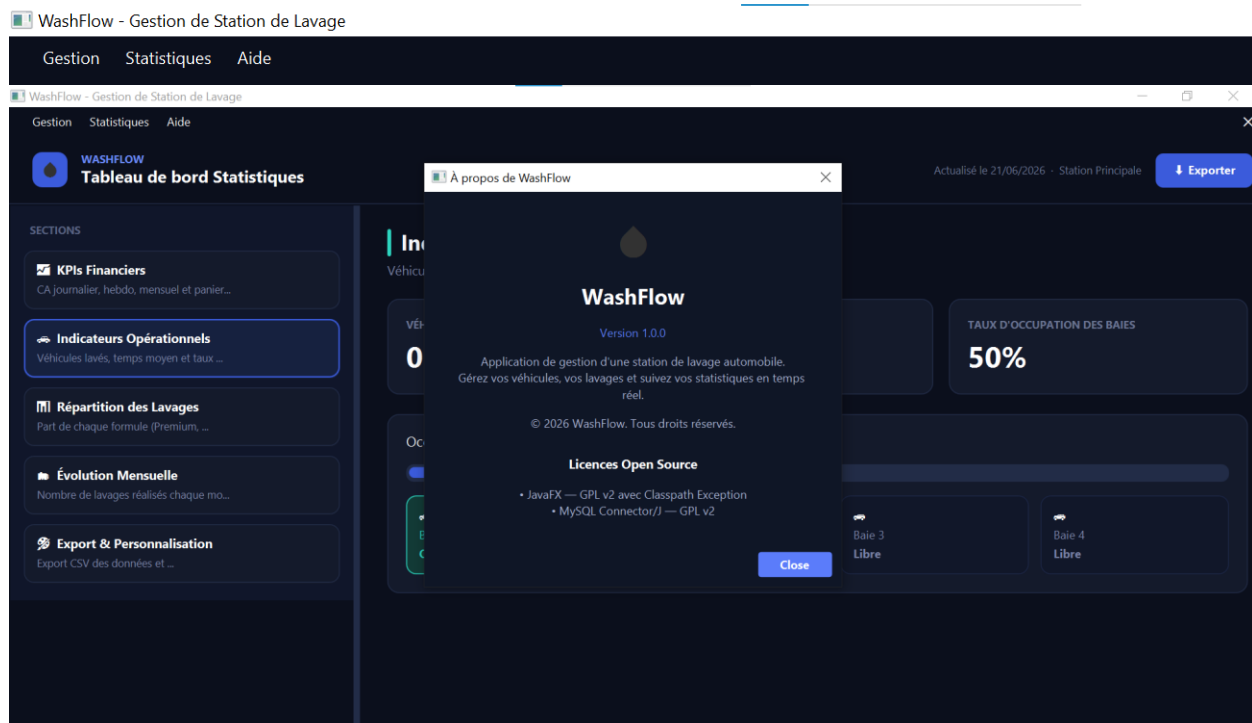
Le tableau de bord est organisé en cinq sections accessibles via un menu latéral : KPIs Financiers (chiffre d'affaires journalier, hebdomadaire, mensuel, panier moyen), Indicateurs Opérationnels (véhicules lavés, temps moyen, taux d'occupation), Répartition des Lavages (graphique en anneau par formule), Évolution Mensuelle (graphique en barres sur 12 mois), et Export & Personnalisation (export CSV et choix de la couleur d'accent).





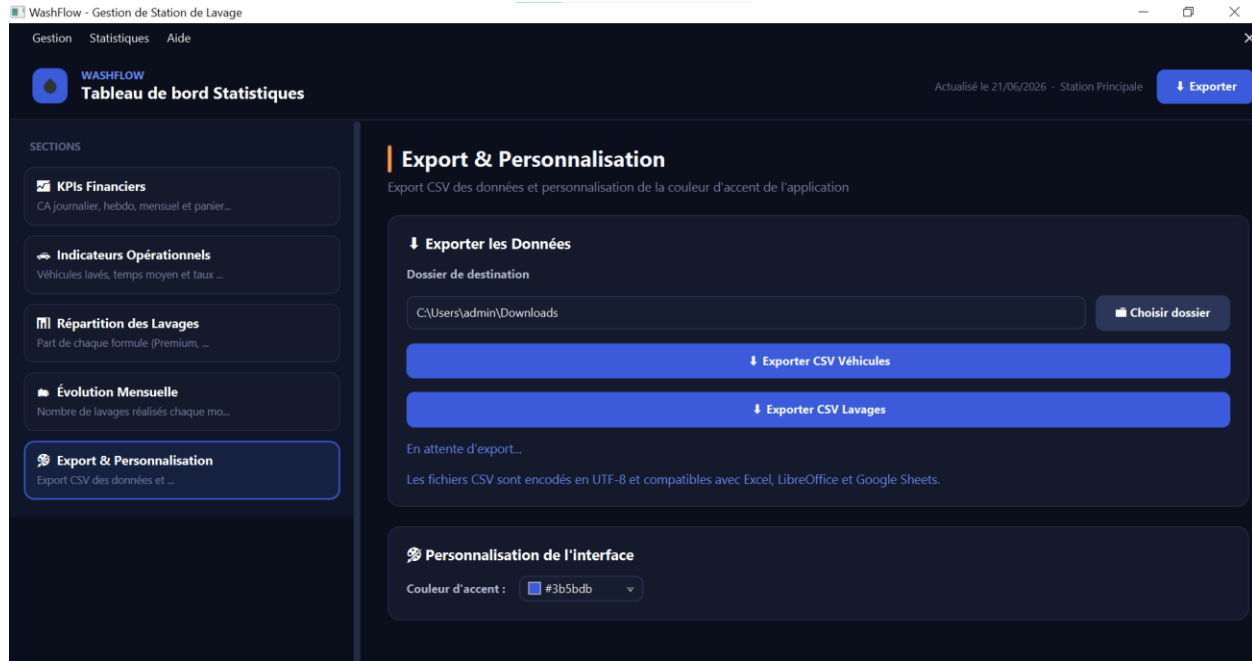
6.5 Menus et dialogues

La barre de menus, présente sur l'ensemble des écrans principaux, propose les menus Gestion (navigation vers les interfaces Véhicules et Lavages), Statistiques (accès au tableau de bord) et Aide (boîte de dialogue « À propos », présentant le nom, la version, une description, le copyright et les licences open source utilisées). Un bouton de fermeture est également disponible directement dans la barre de menus.



6.6 Export de données

Depuis le tableau de bord, l'utilisateur peut exporter l'ensemble des véhicules ou des lavages au format CSV. Un sélecteur de dossier (DirectoryChooser) permet de définir un dossier de destination par défaut, puis un sélecteur de fichier (FileChooser) permet de choisir le nom et l'emplacement précis du fichier généré. Les fichiers produits sont encodés en UTF-8 et compatibles avec Excel, LibreOffice Calc et Google Sheets.



Conclusion

Ce mini-projet nous a permis de mettre en application l'ensemble des concepts abordés durant le module Développement Java IHM : conception d'une interface graphique riche avec JavaFX, organisation du code selon l'architecture MVC complétée par le pattern DAO, persistance des données via JDBC et MySQL, ainsi que la gestion de relations entre entités (véhicule, propriétaire, lavage).

Au-delà des aspects techniques, ce travail nous a sensibilisés à l'importance d'une conception soignée en amont (modélisation UML, structuration de la base de données) pour faciliter le développement et l'évolution ultérieure de l'application. Les principales difficultés rencontrées ont porté sur la gestion cohérente du thème visuel sombre (certains composants JavaFX conservant un comportement de style par défaut) et sur la synchronisation entre les différentes vues lors de la navigation.

Plusieurs pistes d'amélioration pourraient être envisagées pour la suite de ce projet : l'ajout d'une fonctionnalité d'import CSV, la gestion de plusieurs stations de lavage, ou encore la mise en place d'un système de réservation en ligne pour les clients.

Références

- Documentation officielle JavaFX : <https://openjfx.io>
- Documentation MySQL : <https://dev.mysql.com/doc/>
- Documentation Oracle JDBC : <https://docs.oracle.com/javase/tutorial/jdbc/>
- Documentation Apache Maven : <https://maven.apache.org/guides/>
- Connecteur MySQL pour Java (mysql-connector-j) :
<https://dev.mysql.com/doc/connector-j/en/>